



COURSE OBJECTIVES

The course on **Computer Vision** aims to provide the students with the following:



1



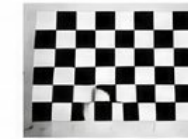
To provide a solid foundation in key concepts related to **image formation**, **feature extraction**, **camera calibration**, and **color models**.



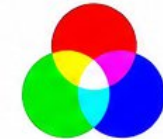
Image Formation



Feature Extraction



Camera Calibration

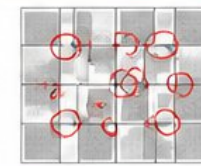


Color Models

2



To understand the key concepts and techniques used in computer vision for detecting and matching **visual features** across images.



3



To provide students with a comprehensive understanding of advanced techniques and methodologies in **image segmentation** and **pattern recognition**.



4



To equip students with the knowledge and skills necessary to utilize the **OpenCV** (Open Source Computer Vision) library effectively for various computer vision tasks.



Object Detection



Feature Detection

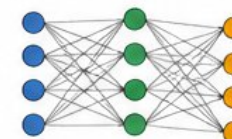


Image Processing

5



To provide students with a comprehensive understanding of **deep learning** techniques and methodologies specifically tailored for solving computer vision problems.



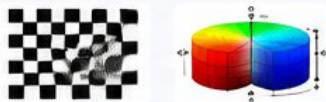
Cat	0.95
Dog	0.03
Bird	0.02





COURSE OUTCOMES

On successful completion of the course, the student will be able to:

1		Explore the fundamentals of computer vision, including image acquisition, camera models and calibration, along with the application of colour models.	Example: Calibrating a camera using checkerboard pattern and converting images from RGB to HSV color space.	
2		Apply feature extraction algorithms to identify and describe important structures and patterns within images, such as edges, corners, and key points.	Example: Detecting edges using Canny edge detector and key points using SIFT or Harris corner detector.	
3		Integrate algorithms, models related to segmenting images into meaningful regions and recognizing patterns.	Example: Segmenting medical images to identify tumors or segmenting roads from aerial images.	
4		Develop applications for object detection and tracking using techniques such as feature detection, descriptors, and matching.	Example: Detecting and tracking vehicles in a video using feature matching and bounding boxes.	
5		Develop deep learning models for various computer vision tasks, including image classification, object detection, and semantic segmentation.	Example: Using CNN to classify images or YOLO model for object detection.	
6		Find solutions for the computer vision problems based on any of the skills acquired throughout the course, ability to find solutions for the problems.	Example: Designing a system to detect defects on a product assembly line.	
7		Develop practical skills through hands-on exercises and projects, including implementing innovative solutions in the field of computer vision.	Example: Building a real-time face recognition system using OpenCV and Python.	
8		Communicate technical concepts and findings related to image formation, feature extraction, camera calibration, and color models through written reports, presentations, and demonstrations.	Example: Preparing a project report and presenting results with graphs, images and key findings.	



SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



UNIT 1: OVERVIEW OF COMPUTER VISION

Computer Vision enables computers to interpret and understand visual information from the world. This unit introduces the fundamental concepts and building blocks of Computer Vision.

TOPICS COVERED IN THIS UNIT

1		Overview of Computer Vision	Introduction to the field of Computer Vision, its goals, and how it enables machines to see and make sense of visual data.
2		Features of Computer Vision	Key capabilities of Computer Vision such as detection, recognition, tracking, segmentation, and understanding visual content.
3		Levels of Computer Vision and Image Processing	Understanding different levels of vision systems, from low-level image processing to high-level semantic understanding.
4		Digital Image Fundamentals	Basics of digital images including pixels, resolution, intensity, and types of images.
5		Image Sensing and Acquisition	How images are captured using sensors and cameras, and the role of lighting, optics, and imaging devices.
6		Sampling and Quantization	Converting a continuous image into a digital form using sampling (discretization) and quantization (intensity levels).
7		Image Formation Models	Mathematical models that explain how 3D scenes are projected onto 2D image planes.
8		Applications of Image Processing	Real-world applications of image processing in various domains such as healthcare, industry, security, and entertainment.
9		Applications of Computer Vision	Use of computer vision in real-world systems like autonomous vehicles, face recognition, robotics, surveillance, and more.



PREPARED BY: **Dr. T.M. USHA**





OVERVIEW OF COMPUTER VISION



Computer Vision is a field of Artificial Intelligence that enables computers to interpret and understand visual information from the world, such as images and videos.

It allows machines to “see”, analyze, and make decisions like human vision.



DEFINITION

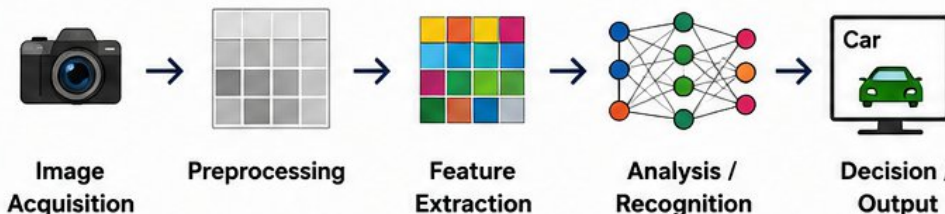
Computer Vision is the science and technology of extracting meaningful information from digital images, videos, and other visual inputs using algorithms and models.



KEY FEATURES

- ✓ Enables machines to interpret visual data
- ✓ Works with images, videos, and real-world scenes
- ✓ Uses algorithms, neural networks, and deep learning
- ✓ Supports automation and intelligent decision making
- ✓ Widely used across industries

HOW COMPUTER VISION WORKS



EXAMPLE

Face Recognition

A system detects and recognizes a person's face from a camera image and matches it with stored data.

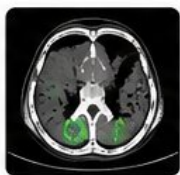


Person ID: 101
Name: Arjun
Match: 98%

APPLICATIONS



Self-Driving Cars



Medical Imaging



Industrial Inspection



Surveillance & Security



Retail Analytics



LEVELS OF COMPUTER VISION



Low-Level Vision

Basic processing like edge detection, filtering, noise removal.



Mid-Level Vision

Detects shapes, textures, objects, and patterns.



High-Level Vision

Understands scenes, objects, and performs reasoning.



IMPORTANCE

- Bridges the gap between human vision and machine understanding.
- Drives automation, accuracy, and efficiency.
- Empowers smart systems to perceive and interact with the real world.

CASE STUDY

Autonomous Vehicle Vision System

Tesla uses computer vision to enable self-driving capabilities.

- ✓ Detects lanes, vehicles, pedestrians in real-time
- ✓ Uses multiple cameras and deep neural networks
- ✓ Helps in navigation, obstacle avoidance, and safe driving
- ✓ Continuously learns and improves from data





SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



FEATURES OF COMPUTER VISION

Computer Vision enables computers to automatically understand and interpret visual information.
It has several important features that make it powerful and widely used.

1. KEY FEATURES OF COMPUTER VISION

	Visual Perception	Enables machines to capture and interpret images and videos like human eyes.	Example: Detecting objects in a traffic scene.
	Object Detection & Recognition	Identifies and recognizes objects, people, or patterns in an image or video.	Example: Face recognition in smartphones.
	Feature Extraction	Extracts useful information (features) such as edges, texture, color, and shape.	Example: Extracting edges from an image.
	Pattern Recognition & Classification	Classifies objects or patterns into different categories based on learned models.	Example: Classifying handwritten digits (0-9).
	Image Segmentation	Divides an image into meaningful regions or segments for better analysis.	Example: Separating road, sky, and vehicles.
	Real-time Processing	Processes and analyzes images or videos instantly, often in real-time.	Example: Live face unlock on mobile devices.
	Automation & Decision Making	Automates tasks and helps in making intelligent decisions without human intervention.	Example: Autonomous vehicles making driving decisions.

4. USEFUL INFORMATION



- ✓ Computer Vision combines image processing, machine learning, and deep learning.
- ✓ OpenCV is the most widely used library for computer vision applications.
- ✓ It is used in industries like healthcare, automotive, security, agriculture, retail, and robotics.
- ✓ Deep learning models like CNN (Convolutional Neural Networks) are widely used for complex visual tasks.

2. EXAMPLES OF FEATURES

Object Detection	Face Recognition	Feature Extraction	Image Segmentation	Pattern Classification
Detecting cars, buses, and bikes	Identifying a person's face	Extracting edges and contours	Segmenting road, sky, and trees	Classifying handwritten digits (0-9)

3. CASE STUDY



Case Study: Smart Retail Analytics

A retail store uses computer vision to analyze customer behavior.

Problem:
Store wants to understand customer footfall, detect shelf interaction, and prevent theft.

Solution:
Cameras with computer vision detect people, track movement, analyze heatmaps, and send alerts for suspicious activity.

Result:
Improved customer experience, optimized store layout, and reduced losses.



5. APPLICATIONS (AT A GLANCE)

Healthcare Detecting diseases, analyzing scans	Automotive Self-driving cars, lane detection	Security Surveillance, face recognition	Retail Customer analytics, shelf monitoring	Agriculture Crop monitoring, disease detection	Robotics Object detection, navigation





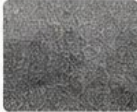
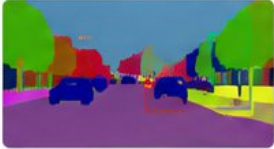
OVERVIEW OF COMPUTER VISION

Computer Vision is a field of Artificial Intelligence that enables computers to interpret and understand visual information from the world, such as images and videos.
It allows machines to "see", analyze, and make decisions like human vision.



LEVELS OF COMPUTER VISION IMAGE PROCESSING

Computer Vision and Image Processing tasks are typically organized into a hierarchy of levels, from low-level operations on pixels to high-level understanding of scenes.

LEVEL	LEVEL NAME	DEFINITION	EXAMPLES	OUTPUT / RESULT	CASE STUDY
1	Low-Level Processing (Pixel Level) 	- Operations performed directly on raw image pixels. - Focuses on improving image quality and extracting basic visual features.	- Image acquisition - Noise removal - Contrast enhancement - Filtering, smoothing - Edge detection	 →  Noisy Image Enhanced Image	Document Scanning Noise removal and contrast enhancement to improve text readability. 
2	Mid-Level Processing (Feature Level) 	- Extracts meaningful features from images. - Identifies edges, corners, textures, shapes, and regions.	- Edge detection - Corner detection - Texture analysis - Morphological operations - Region extraction	   Edge Map Corners Texture	Fingerprint Recognition Feature extraction (ridges and minutiae) is used to identify unique patterns. 
3	High-Level Processing (Object Level) 	- Recognizes and interprets objects in the image. - Involves classification, detection, and tracking.	- Object detection - Classification - Tracking - Scene understanding		Surveillance System Detects and tracks people and vehicles for security monitoring. 
4	Semantic Level (Scene Understanding) 	- Understands the entire scene and context. - Assigns meaning to objects and their relationships.	- Semantic segmentation - Scene classification - Activity recognition 3D understanding	 Road Car Person Tree Sky	Autonomous Driving Understands the scene (road, vehicles, pedestrians, signs) to make driving decisions. 



KEY POINTS



Progression
From pixels to understanding the scene.



Builds Hierarchically
Higher levels depend on results from lower levels.



Uses Various Techniques
Includes filtering, feature extraction, machine learning, and deep learning.



Real-World Impact
Enables machines to perceive, analyze, and act intelligently.

APPLICATION AREAS



Healthcare



Automotive



Security



Industrial



Agriculture



Entertainment

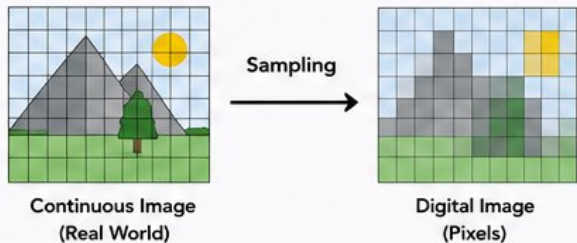




1. A CAMERA CAPTURES AN IMAGE THAT APPEARS BLURRED DUE TO IMPROPER SAMPLING. EXPLAIN HOW SAMPLING AFFECTS IMAGE QUALITY AND SUGGEST A METHOD TO CORRECT IT.

WHAT IS SAMPLING?

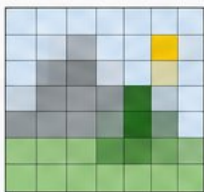
Sampling is the process of converting a continuous image into a digital image by selecting pixels at regular intervals.



- ✓ High Sampling Rate → More pixels → Better image quality
- ✗ Low Sampling Rate → Fewer pixels → Loss of details → Blurred image

WHY DOES BLURRING OCCUR?

- When the sampling rate is too low:
- Fine image details are lost.
- Neighboring pixels merge together.
- Edges become unclear.
- Small objects disappear.
- The image looks blurred or blocky.



KEY POINT

More pixels capture more details.
Better sampling = Better image quality.

EXAMPLE: VEHICLE NUMBER PLATE

ORIGINAL IMAGE (SHARP)

TN 10 AB 1234

CASE 1: IMPROPER SAMPLING (LOW SAMPLING RATE)

Image Size:
50 × 25 pixels



Result:
TN 10 A? 12??

Problems:

- Image is blurred
- Characters hard to read
- OCR accuracy is low

CASE 2: PROPER SAMPLING (HIGH SAMPLING RATE)

Image Size:
400 × 200 pixels

TN 10 AB 1234

Result:
TN 10 AB 1234

Advantages:

- Sharp image
- Details are preserved
- OCR accuracy is high

SAMPLE DATA

Sampling Rate	Image Resolution (Width × Height)	Image Quality	OCR Accuracy (Example)
Low	50 × 25 pixels	Blurred	45%
Medium	200 × 100 pixels	Average	82%
High	400 × 200 pixels	Sharp	99%

CONCLUSION



Improper sampling captures fewer pixels, causing blurred images and loss of important details. Increasing the sampling rate (higher resolution) and using proper image acquisition techniques produces clearer images, improving the accuracy of computer vision tasks.

HOW TO CORRECT IT?



1. Increase Camera Resolution

Capture the image using a higher-resolution camera.
Example: Instead of 640 × 480, use 1920 × 1080.



2. Increase Sampling Rate

Capture more pixels per unit area.
More pixels → Better detail → Clear image.



3. Use Image Interpolation (Software)

Resize the image using:

- Bilinear Interpolation
- Bicubic Interpolation
- Lanczos Interpolation

(Improves appearance but cannot recover lost details.)



4. Improve Camera Focus

Proper focus ensures the sampled pixels represent the scene accurately.

REAL-LIFE APPLICATIONS



Face Recognition

Face cannot be identified correctly



Medical Imaging

Small tumors may be missed



Self-Driving Cars

Road signs may not be detected



QR Code Scanner

QR code cannot be decoded



OCR

Text recognition accuracy decreases



2. GIVEN A REAL-WORLD SCENE CAPTURED UNDER LOW LIGHT, DESCRIBE HOW THE IMAGE SENSING AND ACQUISITION PROCESS INFLUENCES THE FINAL DIGITAL IMAGE.

1 REAL-WORLD SCENE (LOW LIGHT)

A street scene at night captured under low light conditions.



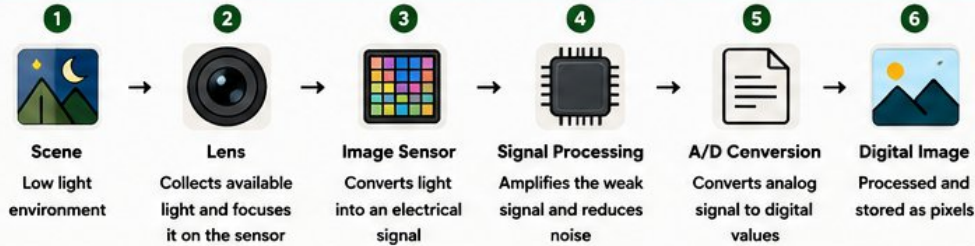
2 WHY LOW LIGHT IS CHALLENGING?

- 1 Less light reaches the camera sensor.
- 2 Sensor captures weak signal.
- 3 More noise is introduced.
- 4 Reduced contrast and details.
- 5 Colors may appear dull or distorted.

3 KEY POINT

- 1 The quality of the final digital image depends on how well the image sensing and acquisition process captures the available light.
- 2 Proper control of camera parameters minimizes noise and improves details.

4 IMAGE SENSING AND ACQUISITION PROCESS (STEP BY STEP)



5 HOW EACH STEP INFLUENCES THE FINAL IMAGE (IN LOW LIGHT)

1	Lens Quality 1 A lens with a large aperture (low f-number like f/1.8) allows more light to reach the sensor. 2 Produces brighter images.	Example: 1 f/1.8 → more light 2 f/5.6 → less light
2	Sensor Sensitivity (ISO) 1 Higher ISO increases sensor sensitivity to light. 2 Makes the image brighter but also increases noise.	Example: 1 ISO 1600 → brighter 2 but noisy image
3	Exposure Time (Shutter Speed) 1 Slower shutter speed allows more light to hit the sensor. 2 Improves brightness but may cause motion blur.	Example: 1 1/30 sec → brighter 2 but may blur
4	Noise in Low Light 1 Low light produces more random variations (noise). 2 Reduces image quality and details.	Example: 1 High noise → grainy 2 image
5	White Balance 1 Under low light (e.g., street lights), color temperature shifts. 2 White balance corrects the colors to look natural.	Example: 1 Without WB → yellowish 2 image
6	Dynamic Range 1 Low light reduces the range between darkest and brightest areas. 2 A good sensor captures more details in shadows.	Example: 1 Better dynamic range → 2 more details in dark areas

6 EXAMPLE RESULT

1 Poor Acquisition (High ISO, Fast Shutter, Small Aperture)	2 Good Acquisition (Optimal ISO, Slow Shutter, Large Aperture)
1 Dark and noisy 2 Less detail 3 Poor color 4 Low visibility	1 Brighter image 2 More details visible 3 Better color 4 Improved visibility

7 SAMPLE ACQUISITION SETTINGS AND EFFECTS

Parameter	1 Low Light Setting (Poor)	2 Optimized Setting (Good)	3 Effect on Image
Aperture (f)	1 f/8 (small)	2 f/1.8 (large)	More light captured
Shutter Speed	1/500 sec (fast)	1/15 sec (slow)	More light captured
ISO	3200 (very high)	800 (moderate)	Less noise, good brightness
White Balance	Auto (incorrect)	Manual (correct)	Natural colors
Sensor Quality	Basic Sensor	High Quality Sensor	Better low light performance
Result	Dark, noisy, low detail image	Bright, clear, high detail image	Better overall image quality

9 REAL-WORLD APPLICATIONS

	1 Surveillance Systems	Clear images help in identifying persons and monitoring activities.
	2 Automotive (Dash Cameras)	Better visibility of roads, sign boards and obstacles.
	3 Medical Imaging	Better image quality for low dose imaging (X-ray, endoscopy).
	4 Astronomy	Capturing faint stars and galaxies.

CASE STUDY: NIGHT SURVEILLANCE CAMERA



1 Problem A security camera captures a warehouse at night. The image is dark and people/objects are not clearly visible.	2 How Image Acquisition Helps 1 Use IR-sensitive sensor 2 Increase exposure time 3 Use proper gain (ISO) 4 Use IR illuminators 5 Apply noise reduction	3 Result Clearer image with visible objects and persons, improving security and monitoring.
--	--	---



10 CONCLUSION

Under low light, the image sensing and acquisition process plays a crucial role in determining the quality of the final digital image. Proper control of lens, exposure, ISO, white balance, and noise reduction leads to brighter, clearer, and more useful images.

11 KEY TAKEAWAY

- | | |
|--------------------------------------|---|
| 1 More light captured → Better image | 3 Correct settings → Good results |
| 2 Less noise → More details | 4 Better image acquisition → Better decisions |

DIGITAL IMAGE FUNDAMENTALS – PHASE 1 (INTRODUCTION)

1. WHAT IS AN IMAGE?

An image is a visual representation of an object or scene.

Examples:



Landscape



Portrait



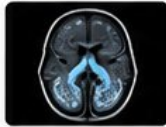
X-Ray



CCTV Image



Satellite Image



Medical Image

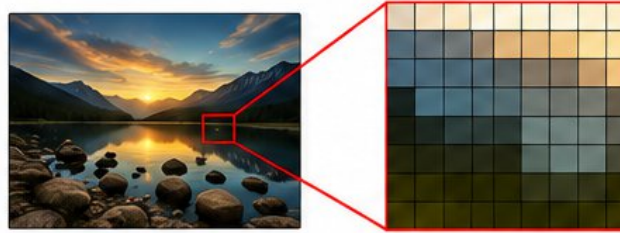


Key Point:

Human eyes see images. Computer stores and processes images.

2. WHAT IS A DIGITAL IMAGE?

A digital image is a 2D array (matrix) of numbers. Each number represents the intensity (gray image) or color (RGB image) of a pixel.



The small squares are called **Pixels**.



Think:

A digital image is made up of thousands or millions of tiny pixels.

3. PIXEL (PICTURE ELEMENT)

Pixel is the smallest unit of a digital image.

Each pixel has a value.

For gray image: 0 (black) to 255 (white)

10	50	80	50	10
20	60	90	60	20
30	70	100	70	30
20	60	90	60	20
10	50	80	50	10

Example:

One Pixel



Value = 80

Pixel values range from 0 to 255.

Higher value → Brighter pixel

4. RESOLUTION

Resolution is the number of pixels in an image. It is represented as Width × Height



Width (Columns)

Height (Rows)

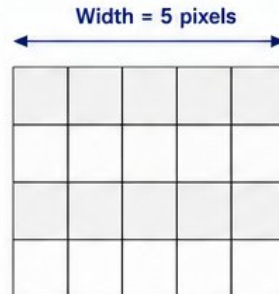
Examples:

- 640 × 480 (Standard)
- 1280 × 720 (HD)
- 1920 × 1080 (Full HD)
- 3840 × 2160 (4K)



More resolution → More pixels → Better image quality

5. IMAGE SIZE (DIMENSIONS)



Example:

Width = 5 pixels
Height = 4 pixels
Image Size = 5 × 4
Total Pixels = 5 × 4 = 20 pixels



Formula:

Total Pixels = Width × Height

6. DISPLAY IMAGE USING PYTHON (OPEN CV)

```
# Install OpenCV: pip install opencv-python
import cv2

# Read an image
img = cv2.imread('sample.jpg')

# Show the image
cv2.imshow('My Image', img)
cv2.waitKey(0) # Wait for key press
cv2.destroyAllWindows()
```

How to Run?

1. Install Python
2. Install OpenCV
3. Save the code in VS Code (e.g., show_image.py)
4. Run the program
5. An image window will open

Output Window



Software / Tools:

- Python
- OpenCV
- VS Code



PHASE 1 LEARNING OUTCOMES:

- ✓ Understand what an image is.
- ✓ Know how a digital image is represented.
- ✓ Understand pixels, resolution and image size.
- ✓ Display an image using Python.

DIGITAL IMAGE FUNDAMENTALS – PHASE 2 (TYPES OF IMAGES)

1. BINARY IMAGE

A binary image has only two possible values:
0 (black) and 1 (white).

Used in: Document scanning, QR codes, OCR,
Fingerprint, Shape analysis.

Example:



Python Code (OpenCV)

```
import cv2
img = cv2.imread('sudoku.png', 0) # Read grayscale
_, binary = cv2.threshold(img, 127, 255,
                          cv2.THRESH_BINARY)
cv2.imshow('Binary Image', binary)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

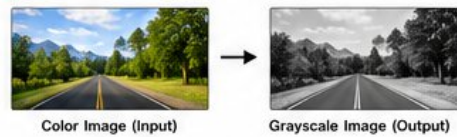
Threshold converts
all pixel values
≥ 127 to white(255),
< 127 to black(0).



2. GRAYSCALE IMAGE

A grayscale image has shades of gray.
Pixel values range from 0 (black) to 255 (white).
Used in: Medical imaging, Face detection,
Edge detection.

Example:



Python Code (OpenCV)

```
import cv2
img = cv2.imread('road.jpg') # Read color image
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow('Grayscale Image', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

Grayscale images
simplify processing
and reduce
computation.



3. RGB COLOR IMAGE

RGB image has 3 channels: Red, Green, Blue.
Each pixel = (R, G, B) values.
Each value ranges from 0 to 255.

Example:



Python Code (OpenCV)

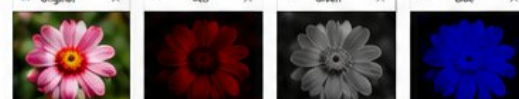
```
import cv2
img = cv2.imread('flower.jpg')
b, g, r = cv2.split(img)
cv2.imshow('Original', img)
cv2.imshow('Red', r)
cv2.imshow('Green', g)
cv2.imshow('Blue', b)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Note:

Splitting channels
helps in many
vision tasks.



Output:



4. COLOR CHANNELS (RGB)

A color image can be viewed as 3 matrices
(R, G, B channels).

Example:

100	150	200
50	100	150
0	50	100

=

100	150	200
50	100	150
0	50	100

R Channel
(Red)

100	150	200
50	100	150
0	50	100

G Channel
(Green)

100	150	200
50	100	150
0	50	100

B Channel
(Blue)

Python Code (OpenCV)

```
import cv2
img = cv2.imread('flower.jpg')
b, g, r = cv2.split(img)
print('R channel shape:', r.shape)
print('G channel shape:', g.shape)
print('B channel shape:', b.shape)
```

Output (Console):

```
R channel shape: (600, 800)
G channel shape: (600, 800)
B channel shape: (600, 800)
```

Note:

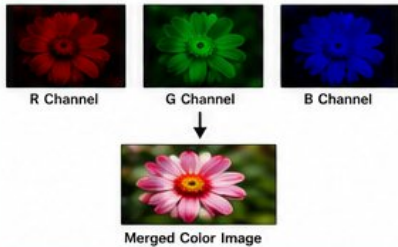
Each channel
has the same
size as the
original image.



5. MERGING CHANNELS

We can merge separate R, G, B channels
to form a color image.

Example:



Python Code (OpenCV)

```
import cv2
b, g, r = cv2.split(cv2.imread('flower.jpg'))
merged = cv2.merge([b, g, r])
cv2.imshow('Merged Image', merged)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

Merging combines
B, G, R channels
to recreate the
original color
image.



6. COLOR TO GRAY TO BINARY

A practical workflow: Color → Grayscale → Binary
Used in many real-world applications.

Example:



Python Code (OpenCV)

```
import cv2
img = cv2.imread('flower.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255,
                          cv2.THRESH_BINARY)
cv2.imshow('Color', img)
cv2.imshow('Grayscale', gray)
cv2.imshow('Binary', binary)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

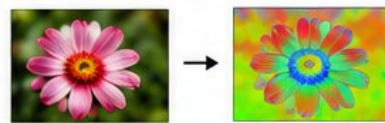
This conversion
simplifies the image
and makes it
easier to analyze.



7. COLOR SPACE CONVERSION

Images can be represented in different color spaces.
Examples: BGR, RGB, HSV, LAB, YCrCb.

Example (BGR to HSV):



Python Code (OpenCV)

```
import cv2
img = cv2.imread('flower.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
cv2.imshow('BGR Image', img)
cv2.imshow('HSV Image', hsv)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

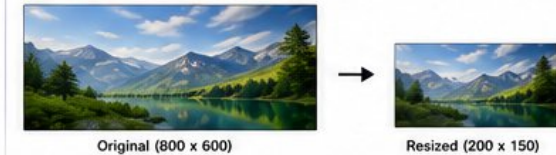
HSV is useful
for color-based
segmentation.



8. IMAGE SAMPLING (RESIZING)

Sampling changes the size (number of pixels)
of an image.

Example:



Python Code (OpenCV)

```
import cv2
img = cv2.imread('landscape.jpg')
resized = cv2.resize(img, (200, 150))
cv2.imshow('Original', img)
cv2.imshow('Resized', resized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Note:

More pixels =
better quality.
Fewer pixels =
smaller size but
less detail.



KEY TAKEAWAY: Understanding different types of images and how to convert between them is the foundation of Computer Vision.

✓ Binary for shapes

✓ Grayscale for intensity

✓ RGB for colors

✓ Channels for analysis

✓ Conversions for better results.

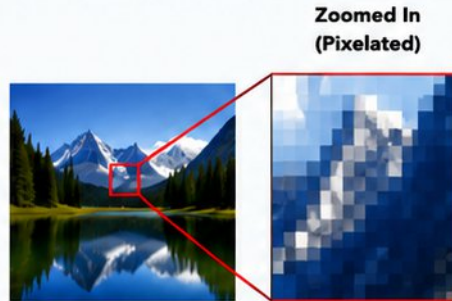
Free, high-quality, copyright-safe images. Can ownload any image from these sites & use it with OpenCV programs

S.No	Website	Best For
1	Unsplash – https://unsplash.com	Nature, flowers, roads, landscapes, people, buildings
2	Pexels – https://www.pexels.com	Free high-quality photos and videos
3	Pixabay – https://pixabay.com	Photos, illustrations, vectors, icons
4	Burst by Shopify – https://burst.shopify.com	Business and everyday objects
5	Freepik (Free Images) – https://www.freepik.com	Graphics and photos (check license terms)
6	Wikimedia Commons – https://commons.wikimedia.org	Educational images, maps, documents, medical images
7	Open Images Dataset – https://storage.googleapis.com/openimages/web/index.html	Large object detection dataset
8	Kaggle Datasets – https://www.kaggle.com/datasets	Computer Vision datasets
9	Roboflow Universe – https://universe.roboflow.com	Object detection and segmentation datasets
10	OpenCV Sample Images – https://github.com/opencv/opencv/tree/4.x/samples/data	Official OpenCV sample images
11	CVLIBS Dataset Collection – https://www.cvlibs.net/datasets/	Many Computer Vision datasets
12	TensorFlow Datasets – https://www.tensorflow.org/datasets	Ready-to-use ML/CV datasets
13	ImageNet – https://www.image-net.org	Image classification dataset
14	COCO Dataset – https://cocodataset.org	Object detection, segmentation, captions
15	MNIST – https://yann.lecun.com/exdb/mnist/	Handwritten digits
16	Fashion-MNIST – https://github.com/zalandoresearch/fashion-mnist	Clothing images
17	CIFAR-10 – https://www.cs.toronto.edu/~kriz/cifar.html	Basic image classification
18	Oxford Flowers 102 – https://www.robots.ox.ac.uk/~vgg/data/flowers/102/	Flower classification
19	Caltech 101 – https://data.caltech.edu/records/mzrjq-6wc02	Object recognition
20	CelebA – https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html	Face recognition

Q27. AN IMAGE APPEARS PIXELATED WHEN ZOOMED. ANALYZE THE ROLE OF SAMPLING AND RESOLUTION IN THIS EFFECT.

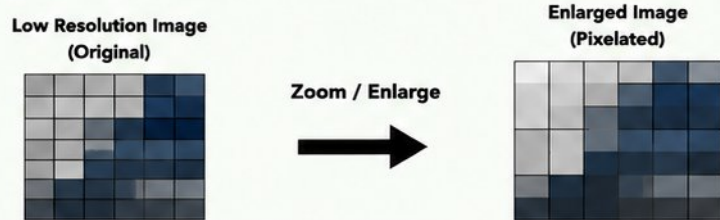
1. PROBLEM EXPLANATION

When a digital image is zoomed in (enlarged), it appears **blocky or pixelated**. This happens because the image is made of a **fixed number of pixels**. When we zoom beyond the original resolution, **each pixel becomes larger** and the gaps between pixel values become visible as square blocks.



2. WHY DOES PIXELATION HAPPEN?

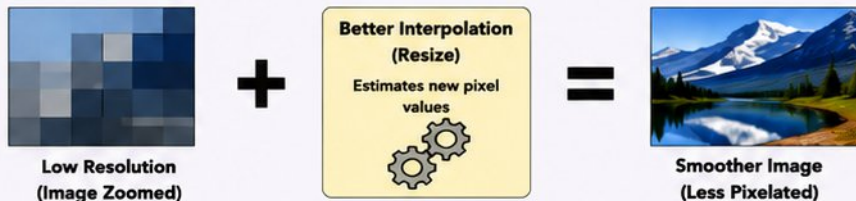
- An image is created using a **fixed number of pixels (sampling)**.
- When we zoom beyond the original resolution, there are **not enough pixels** to represent new details.
- The system just enlarges existing pixels, making them visible as **square blocks**.
- This is a limitation of **low sampling (low resolution)**.



3. SOLUTION

To reduce pixelation when zooming:

- ✓ Use **higher resolution** images (more pixels).
- ✓ Use **interpolation techniques** while resizing (e.g., **INTER_CUBIC** or **INTER_LANCZOS4**).
- ✓ These methods estimate new pixel values and create **smoother results**.



KEY TAKEAWAY

Pixelation occurs due to low sampling / low resolution. Use higher resolution images and proper interpolation methods (e.g., **INTER_CUBIC**) to reduce pixelation when zooming.

4. PYTHON CODE (OpenCV)

```
import cv2
import numpy as np

# Read image
img = cv2.imread('landscape.jpg')

# Zoom factor
scale = 3 # enlarge 3 times

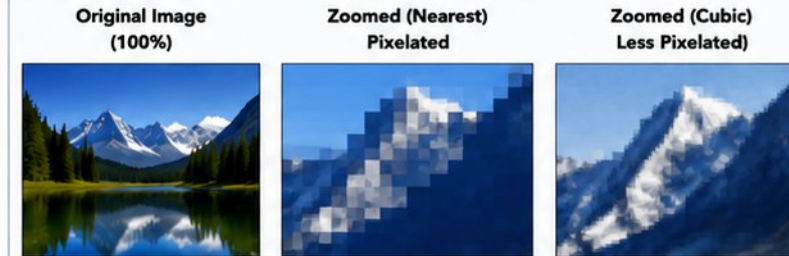
# 1) Naive zoom (pixelated)
zoom_pixelated = cv2.resize(img, None,
                             fx=scale, fy=scale,
                             interpolation=cv2.INTER_NEAREST)

# 2) Better zoom (less pixelated)
zoom_better = cv2.resize(img, None,
                         fx=scale, fy=scale,
                         interpolation=cv2.INTER_CUBIC)

# Save outputs
cv2.imwrite('zoom_pixelated.jpg', zoom_pixelated)
cv2.imwrite('zoom_better.jpg', zoom_better)

# Show (optional)
cv2.imshow('Original', img)
cv2.imshow('Pixelated Zoom (Nearest)', zoom_pixelated)
cv2.imshow('Better Zoom (Cubic)', zoom_better)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

5. OUTPUT



OBSERVATION

- Nearest neighbour zooming shows visible blocks (pixelation).
- Cubic interpolation produces smoother edges and less pixelation.



TRY IT YOURSELF

Download any low resolution image, zoom it using different interpolation methods and compare the results.

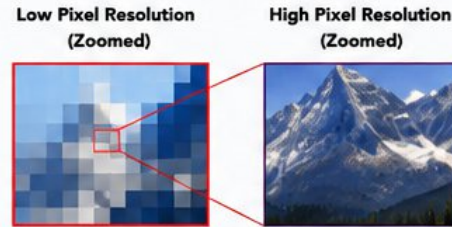
Q4. Differentiate how pixel resolution and intensity resolution affect image representation in a practical application like medical imaging.

1. PROBLEM EXPLANATION

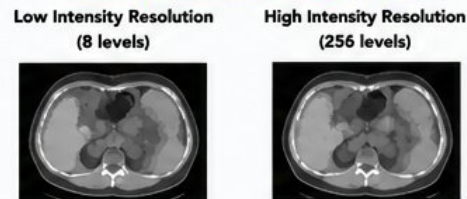
Image quality depends on two important resolutions:

- **Pixel Resolution (Spatial Resolution):**
Number of pixels (width × height) in an image. It controls the level of detail. Low pixel resolution causes pixelation (blocky effect) when zoomed.
- **Intensity Resolution (Gray Level Resolution):**
Number of intensity levels (bit depth) used to represent pixel values. It controls how smooth the brightness variations are. Low intensity resolution causes banding (posterization).

Pixel Resolution (Spatial)



Intensity Resolution (Gray Level)



2. PRACTICAL EXPLANATION IN MEDICAL IMAGING



Pixel Resolution

In medical images (CT, MRI, X-ray), higher pixel resolution captures smaller structures and fine details (tumors, vessels). Low pixel resolution → loss of detail → important information may be missed.



Intensity Resolution

Higher intensity resolution (e.g., 12-bit, 16-bit) provides more gray levels to represent subtle differences in tissue density. Low intensity resolution → banding → loss of subtle contrast between tissues.

Conclusion:

For accurate diagnosis, medical imaging needs both high pixel resolution (to see tiny details) and high intensity resolution (to see subtle differences).

3. SOLUTION APPROACH

1. Load an image.
2. Create a low pixel resolution version and zoom it → shows pixelation.
3. Keep pixel resolution high and reduce intensity levels → shows banding.
4. Display and compare results.

6. KEY TAKEAWAYS

- ✓ Low pixel resolution → pixelation (blocky effect).
- ✓ Low intensity resolution → banding (loss of smoothness).
- ✓ In medical imaging, both high pixel resolution and high intensity resolution are essential for accurate diagnosis.

4. PYTHON CODE (OpenCV)

```
import cv2
import numpy as np

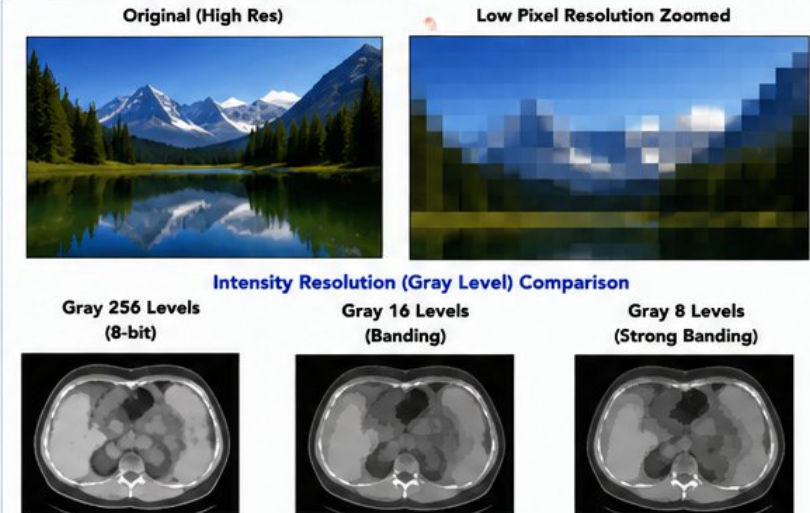
# Load image (any high quality image)
img = cv2.imread('landscape.jpg')

# ----- Pixel Resolution Effect -----
# Low pixel resolution (downsample)
low_res = cv2.resize(img, (160, 120)) # very low resolution
# Zoom back to original size using nearest (shows pixelation)
low_res_zoom = cv2.resize(low_res, (img.shape[1], img.shape[0]),
                           interpolation=cv2.INTER_NEAREST)

# ----- Intensity Resolution Effect -----
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Reduce intensity levels to 8 (3-bit -> 8 levels)
gray_8level = (gray // 32 * 32).astype(np.uint8)
# Reduce intensity levels to 16 (4-bit -> 16 levels)
gray_16level = (gray // 16 * 16).astype(np.uint8)

# Show all results
cv2.imshow('Original (High Res)', img)
cv2.imshow('Low Pixel Resolution Zoomed', low_res_zoom)
cv2.imshow('Gray 256 Levels (8-bit)', gray)
cv2.imshow('Gray 16 Levels (Banding)', gray_16level)
cv2.imshow('Gray 8 Levels (Strong Banding)', gray_8level)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

5. OUTPUT (What You Will See)



7. TRY IT YOURSELF

- Try different downsample sizes (64×48, 320×240) and zoom them.
- Reduce intensity levels to 4, 8, 16, 32 and observe banding.
- Use CT or MRI images for better understanding.



Tip:
Use `INTER_NEAREST` to clearly see pixelation. Try `INTER_LINEAR` to observe smoother (but less blocky) zoom.

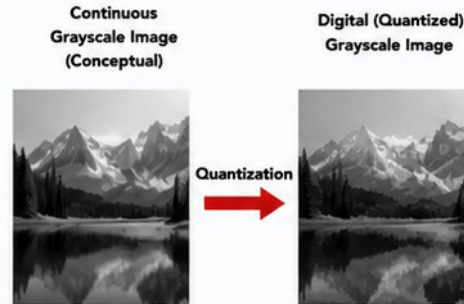
Q7. A GRAYSCALE IMAGE IS CONVERTED INTO A DIGITAL IMAGE. DESCRIBE THE ROLE OF QUANTIZATION IN THIS PROCESS AND ITS IMPACT ON IMAGE DETAIL.

1. PROBLEM EXPLANATION

When a continuous grayscale image is converted into a digital image, its pixel intensities (brightness values) are quantized into a finite number of discrete levels. This process is called **Quantization**.

It reduces the number of possible intensity values (from 0–255 to fewer levels) depending on the bit depth used.

Quantization affects how much detail, smoothness, and subtle variations are preserved in the image.



2. ROLE OF QUANTIZATION

- Converts continuous intensity values into L discrete levels.
- The number of levels $L = 2^b$, where b = number of bits (bit depth).
Example: $b = 3$ bits $\rightarrow L = 2^3 = 8$ levels (0–7)
- Each pixel value is mapped to the nearest available level.
- Enables digital storage and processing (computers can store only discrete values).

Example (3-bit Quantization)

0	32	64	96	128	160	192	224	255
Level: 0	1	2	3	4	5	6	7	

Any pixel value between 0–31 $\rightarrow$ 0, 32–63 $\rightarrow$ 1, ..., 224–255 $\rightarrow$ 7

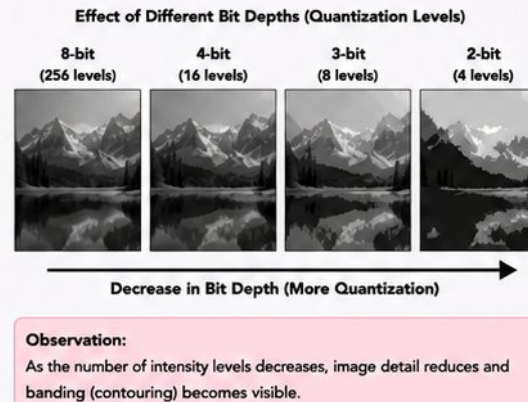
3. IMPACT ON IMAGE DETAIL

✓ High Quantization (More Bits)

- More intensity levels (e.g., 8-bit $\rightarrow$ 256 levels).
- Smooth transitions.
- Preserves fine details and subtle variations.
- Larger file size.

✗ Low Quantization (Less Bits)

- Fewer intensity levels (e.g., 2-bit $\rightarrow$ 4 levels).
- Causes banding (posterization).
- Loses subtle details, smooth gradients.
- Smaller file size but degraded quality.



5. OUTPUT (What You Will See)



Key Takeaway:

Quantization is essential for converting continuous grayscale values into digital form, but lower bit depth reduces image detail and introduces banding.

4. PYTHON CODE (OpenCV)

```
import cv2
import numpy as np

# Read grayscale image
img = cv2.imread('lena_gray.png', cv2.IMREAD_GRAYSCALE)

# Quantization function
def quantize(img, bits):
    levels = 2 ** bits  # Number of quantization levels
    factor = 256 // levels  # Step size
    quantized = (img // factor) * factor  # Map to nearest level
    return quantized

# Create quantized images
img_8 = quantize(img, 8)  # 8-bit (256 levels) - original
img_4 = quantize(img, 4)  # 4-bit (16 levels)
img_3 = quantize(img, 3)  # 3-bit (8 levels)
img_2 = quantize(img, 2)  # 2-bit (4 levels)

# Save results
cv2.imwrite('q7_8bit.png', img_8)
cv2.imwrite('q7_4bit.png', img_4)
cv2.imwrite('q7_3bit.png', img_3)
cv2.imwrite('q7_2bit.png', img_2)

# Display results
titles = ['8-bit (256 levels)', '4-bit (16 levels)',
          '3-bit (8 levels)', '2-bit (4 levels)']
images = [img_8, img_4, img_3, img_2]
for i in range(4):
    plt.subplot(1,4,i+1)
    plt.imshow(images[i], cmap='gray', vmin=0, vmax=255)
    plt.title(titles[i], fontsize=10)
    plt.axis('off')
plt.tight_layout(); plt.show()
```

6. TRY IT YOURSELF

- Try different bit depths: 8, 6, 5, 4, 3, 2 bits.
- Observe how the number of levels affects quality and banding.
- Use your own grayscale image and compare the results.

Tip:

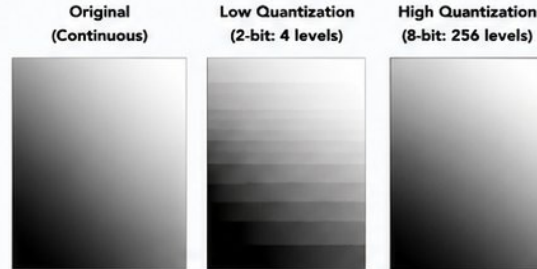
More bits $\rightarrow$ more levels $\rightarrow$ more details but larger storage size.

Q22. A Digital Camera Produces Images with Banding Effects. Analyze How Quantization Levels Contribute to This Issue and Propose a Solution.

1. PROBLEM EXPLANATION

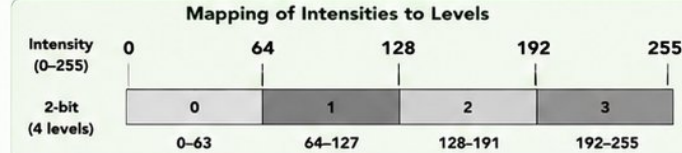
Banding (posterization) appears as visible steps or stripes in areas of smooth intensity change (e.g., sky, gradient backgrounds). It occurs because the image uses too few quantization levels (low bit depth) to represent the large range of brightness values. Each level represents a wide range of intensities, causing sudden jumps between levels.

Example: Smooth Gradient



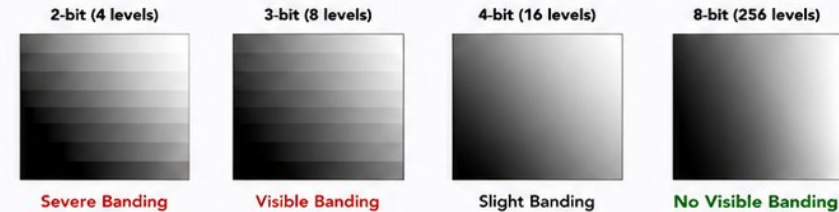
2. HOW QUANTIZATION CAUSES BANDING

- Image intensities are quantized into $L = 2^b$ levels.
- Low b (bit depth) $\rightarrow$ small $L \rightarrow$ large gap between levels.
- The eye perceives these large gaps as visible bands.
- High $b \rightarrow$ large $L \rightarrow$ small gaps $\rightarrow$ smooth appearance.



- ❌ Low levels $\rightarrow$ wide ranges $\rightarrow$ visible bands
- ✅ High levels $\rightarrow$ narrow ranges $\rightarrow$ smooth tones

3. IMPACT OF DIFFERENT QUANTIZATION LEVELS (BIT DEPTH)



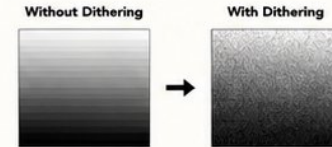
Observation:
As bit depth increases, number of quantization levels increases, so the bands become less visible and the image looks smoother.

4. SOLUTION

- ✅ Use higher bit depth (e.g., 8-bit, 10-bit, 12-bit).
- ✅ Apply dithering (adds tiny noise) to mask banding when bit depth is low.
- ✅ Use proper exposure and quantization during image acquisition.
- ✅ For storage efficiency, use higher bit depth for important images (medical, HDR, etc.).

What is Dithering?

Dithering adds small random variations (noise) to pixel values before quantization so that band boundaries are less noticeable.



5. PYTHON CODE (OpenCV + NumPy)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a smooth grayscale gradient (0 to 255)
gradient = np.tile(np.linspace(0, 255, 256, dtype=np.uint8), (256, 1))

# Quantization function
def quantize(img, bits):
    levels = 2 ** bits
    factor = 256 // levels
    quantized = (img // factor) * factor
    return quantized

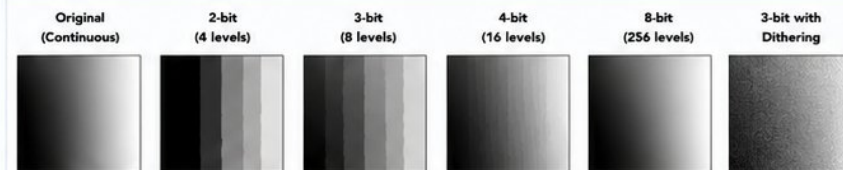
# Dithering using random noise (Floyd-Steinberg approximation not used for simplicity)
def add_dither(img, noise_level=8):
    noise = np.random.randint(-noise_level, noise_level+1, img.shape, dtype=np.int16)
    noisy_img = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)
    return noisy_img

# Original gradient
gradient = np.tile(np.linspace(0, 255, 256, dtype=np.uint8), (256, 1))

# Quantized images with different bit depths
img_2bit = quantize(gradient, 2)
img_3bit = quantize(gradient, 3)
img_4bit = quantize(gradient, 4)
img_8bit = quantize(gradient, 8)
img_dithered = quantize(add_dither(gradient, noise_level=8), 3)

# Display results
images = (gradient, img_2bit, img_3bit, img_4bit, img_8bit, img_dithered)
titles = ["Original (Continuous)", "2-bit (4 levels)", "3-bit (8 levels)", "4-bit (16 levels)", "8-bit (256 levels)", "3-bit with Dithering")
plt.figure(figsize=(18, 4))
for i in range(len(images)):
    plt.subplot(1, len(images), i+1)
    plt.imshow(images[i], cmap='gray', vmin=0, vmax=255)
    plt.title(titles[i], fontsize=8)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

6. OUTPUT (What You Will See)



Explanation:

- 2-bit (4 levels) $\rightarrow$ large steps $\rightarrow$ strong banding.
- 3-bit (8 levels) $\rightarrow$ visible banding.
- 4-bit (16 levels) $\rightarrow$ slight banding.
- 8-bit (256 levels) $\rightarrow$ smooth gradient (no visible banding).
- 3-bit with dithering $\rightarrow$ banding is masked by noise $\rightarrow$ appears smoother.

7. KEY TAKEAWAYS

- ✅ Low quantization levels (low bit depth) cause banding or posterization.
- ✅ Higher quantization levels preserve smooth intensity transitions.
- ✅ Dithering is an effective technique to reduce visible banding in low bit-depth images.
- ✅ Always choose appropriate bit depth based on application (e.g., medical imaging needs higher bit depth).

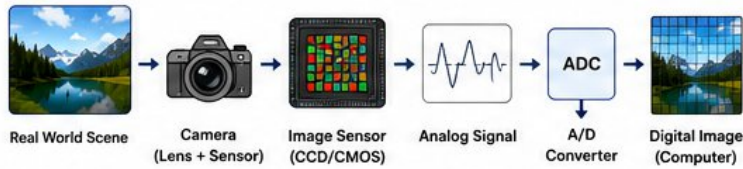
★ TRY IT YOURSELF

Change bit depths (2, 3, 4, 5, 6, 8, 10, 12 bits) and observe banding. Try different noise levels in dithering and compare results.

IMAGE SENSING & ACQUISITION, SAMPLING AND QUANTIZATION – PYTHON EXAMPLES

1. IMAGE SENSING & ACQUISITION

Capture an image from camera (sensing) or load from file (acquisition).



Python Code: Image Acquisition

```
import cv2

# Read image from file
img = cv2.imread('landscape.jpg')
if img is None:
    print("Image not found. Check the path.")
    exit()

# Convert BGR (OpenCV default) to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Display the image
cv2.imshow('Acquired Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output

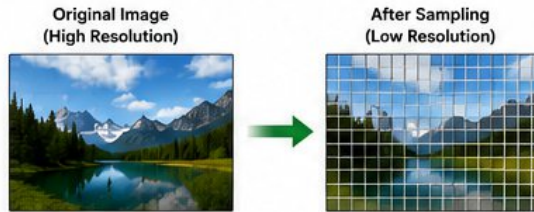


Explanation

- The camera sensor converts light into electrical (analog) signals.
- The ADC converts analog signals into digital values.
- The digital image is stored in the computer.

2. SAMPLING (Spatial Resolution)

Sampling converts a continuous image into a grid of pixels. More samples (pixels) → better detail.



Python Code: Sampling (Downsampling)

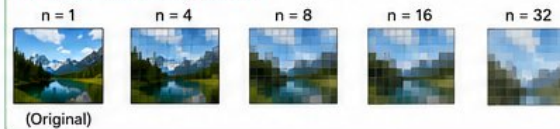
```
import cv2
import numpy as np

# Read high resolution image
img = cv2.imread('landscape.jpg')

# Downsampling by taking every nth pixel
n = 8 # Sampling factor (increase n → lower resolution)
sampled_img = img[::n, ::n]

# Display results
cv2.imshow('Original (High Res)', img)
cv2.imshow(f'Sampled (n={n})', sampled_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Try Different Sampling Factors



Key Point

Higher sampling rate (smaller n) → more pixels → better spatial resolution.

3. QUANTIZATION (Intensity Resolution)

Quantization assigns discrete intensity values to each pixel. Higher bit depth → more intensity levels → better quality.



Python Code: Quantization (Reduce Bit Depth)

```
import cv2
import numpy as np

# Read grayscale image
gray = cv2.imread('landscape.jpg', cv2.IMREAD_GRAYSCALE)

# Quantization function
def quantize(image, bits):
    levels = 2 ** bits # Number of intensity levels
    factor = 256 // levels # Quantization step
    return (image // factor) * factor

# Apply quantization for different bit depths
quant_8bit = quantize(gray, 8) # Original (256 levels)
quant_4bit = quantize(gray, 4) # 16 levels
quant_2bit = quantize(gray, 2) # 4 levels
quant_1bit = quantize(gray, 1) # 2 levels (Binary)

# Display results
cv2.imshow('Original 8-bit', quant_8bit)
cv2.imshow('4-bit (16 levels)', quant_4bit)
cv2.imshow('2-bit (4 levels)', quant_2bit)
cv2.imshow('1-bit (2 levels)', quant_1bit)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

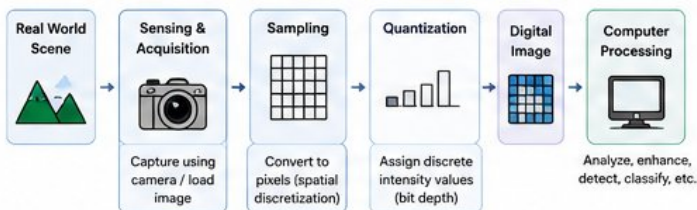
Quantization Results



Key Point

Higher bit depth → more intensity levels → smoother image quality.

COMPLETE IMAGE ACQUISITION PIPELINE IN PYTHON



Pipeline Code (All Steps Together)

```
import cv2
import numpy as np

# 1. Acquisition
img = cv2.imread('landscape.jpg')

# 2. Sampling (Downsample by 4)
sampled = img[::4, ::4]

# 3. Quantization to 4-bit
gray = cv2.cvtColor(sampled, cv2.COLOR_BGR2GRAY)
quant_4bit = (gray // 16) * 16

# 4. Show final result
cv2.imshow('Final (Sampled + 4-bit Quantized)', quant_4bit)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

SUMMARY TABLE

Concept	What it Means	Python Operation	Effect
Image Sensing & Acquisition	Capture real world scene and convert to digital image	<code>cv2.imread()</code>	Get image from camera or file
Sampling (Spatial Resolution)	Divide continuous image into pixels	<code>img[::n, ::n]</code>	More pixels → better detail
Quantization (Intensity Resolution)	Assign discrete intensity values to each pixel	Reduce bit depth (e.g., to 8, 4, 2, 1 bits)	More levels → better quality



Remember:

Sampling → Size (number of pixels)

Quantization → Quality (intensity levels)

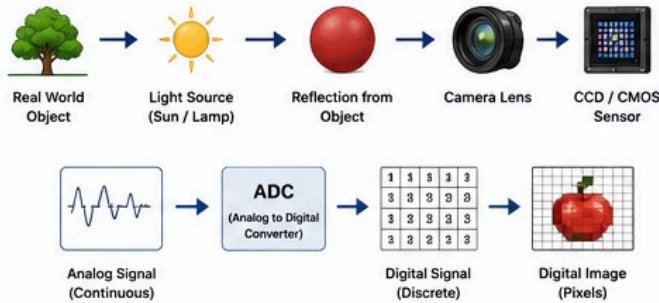
These are the foundation steps in Digital Image Processing & Computer Vision.

IMAGE FORMATION MODELS IN COMPUTER VISION

Concept, Model, Pipeline and Python Implementation

1. WHAT IS IMAGE FORMATION?

Image formation is the process of capturing a real world scene and converting it into a digital image that can be processed by a computer.



The camera captures light reflected by objects, converts it into electrical signals, digitizes the signals and produces a digital image.

2. IMAGE FORMATION MODEL

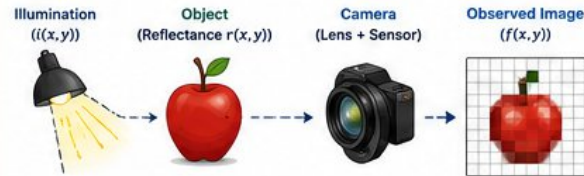
The most widely used mathematical model in computer vision is:

$$f(x, y) = i(x, y) \times r(x, y)$$

$f(x, y)$ → Observed Image (what the camera captures)

$i(x, y)$ → Illumination (light intensity falling on the object)

$r(x, y)$ → Reflectance (amount of light reflected by the object)



The observed image depends on how much light falls on the object (illumination) and how much the object reflects that light (reflectance).

3. PYTHON IMPLEMENTATION (IMAGE ACQUISITION)

We use OpenCV library in Python to read, display and save an image.

```
# Import OpenCV
import cv2

# Read an image from file
img = cv2.imread('flower.jpg')

# Check if image is loaded
if img is None:
    print('Error: Image not found!')
    exit()

# Display the image
cv2.imshow('Input Image', img)

# Print image properties
print('Image Shape (H, W, C):', img.shape)
print('Height      : ', img.shape[0], 'pixels')
print('Width       : ', img.shape[1], 'pixels')
print('Channels     : ', img.shape[2])
print('Data Type    : ', img.dtype)

# Save the image
cv2.imwrite('acquired_image.png', img)
print('Image saved as 'acquired_image.png')

# Wait for key press and close windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation

- `cv2.imread()` → reads the image from file.
- `cv2.imshow()` → displays the image.
- `img.shape` → gives (height, width, channels).
- `cv2.imwrite()` → saves the image.
- `waitKey(0)` → waits until any key is pressed.
- `destroyAllWindows()` → closes all windows.

4. INPUT IMAGE

We will use the following sample image.

File Name: **flower.jpg**



Image Properties

Resolution : 1024 × 768
Height : 768 pixels
Width : 1024 pixels
Channels : 3 (RGB)
Data Type : uint8
Total Pixels : 786,432

This is a color image (RGB) where each pixel has 3 values (R, G, B) in the range [0, 255].

5. OUTPUT

Input Image (Original)

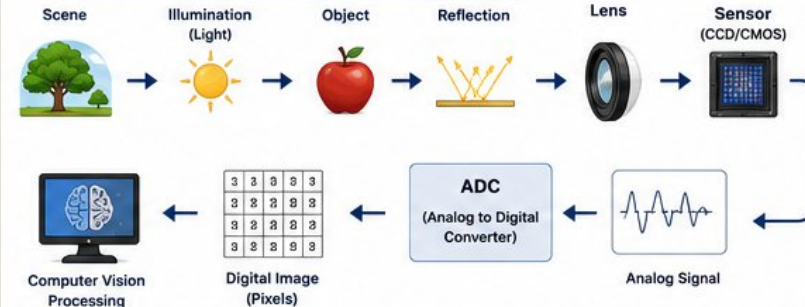


Output Image (Captured Digital Image)



- ✓ Image is successfully acquired from the file.
- ✓ It is converted into a matrix of pixels.
- ✓ Each pixel contains intensity values.
- ✓ The digital image is now ready for computer vision processing.

6. COMPLETE IMAGE FORMATION PIPELINE



8. STUDENT ACTIVITY / TASK

Implement the image formation concept and explore an image using Python.

Your Task:

- Read any image using OpenCV.
- Display the image.
- Print the following details:
 - Height
 - Width
 - Number of Channels
 - Data Type
- Save the image with a new name.
- Explain the image formation model $f(x, y) = i(x, y) \times r(x, y)$ in your own words.
- Submit your Python code and screenshots of input & output.

Sample Run (Example Output in Console)

```
Image Shape (H, W, C): (768, 1024, 3)
Height      : 768 pixels
Width       : 1024 pixels
Channels     : 3
Data Type   : uint8
Image saved as 'acquired_image.png'
(Press any key to exit)
```

Files Generated

acquired_image.png



Learning Outcomes

- Understand how images are formed in a camera.
- Learn the mathematical model of image formation.
- Implement image acquisition using Python.
- Explore image properties and save output.
- Build the foundation for advanced Computer Vision applications.

7. APPLICATIONS



Smartphone Cameras



Self-Driving Cars



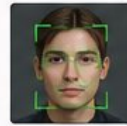
Satellite Imaging



Medical Imaging



Robotics & Automation



Face Recognition

Key Takeaway:

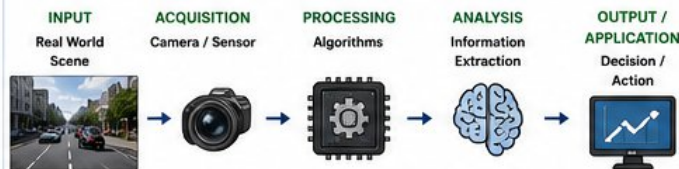
An image is formed when light from a scene is reflected by objects and captured by a sensor. The mathematical model $f(x, y) = i(x, y) \times r(x, y)$ explains how illumination and reflectance produce the observed image.

APPLICATIONS OF IMAGE PROCESSING AND COMPUTER VISION

Solving Real-World Problems and Improving Human Life

1. OVERVIEW

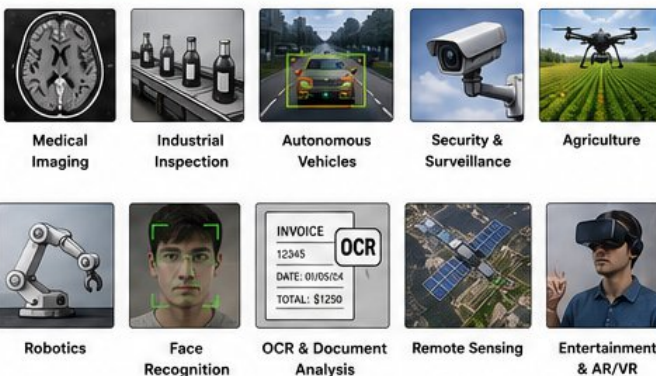
Image Processing improves images (enhancement, restoration, compression)
Computer Vision understands images and makes decisions.



Benefits

- Automates tasks
- Improves accuracy
- Saves time and cost
- Helps in better decision making

2. MAJOR APPLICATION AREAS



3. COMMON IMAGE PROCESSING & CV TECHNIQUES

Category	Techniques	Example
Image Enhancement	Histogram Equalization, Filtering, Sharpening	  Before After
Image Restoration	Noise Removal, Deblurring	  Noisy Image Restored Image
Image Segmentation	Thresholding, Edge Detection, Region Growing	  Original Segmented
Feature Extraction	Corners, Edges, Texture, SIFT, SURF	 Keypoints
Pattern Recognition	Classification, Object Detection, Deep Learning	 Detected Objects

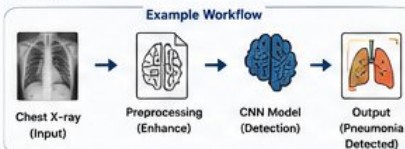
4. CASE STUDIES: REAL-TIME PROBLEMS SOLVED

CASE STUDY 1: DISEASE DETECTION FROM MEDICAL IMAGES

Problem: Manual analysis of X-ray/CT images is time consuming and error-prone.

Solution: Image processing and deep learning are used to detect diseases (e.g., pneumonia, tumors).

Impact: Early detection, accurate diagnosis, saves lives.



Real-World Use:

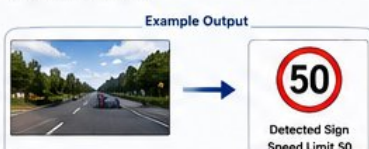
AI systems in hospitals assist radiologists and reduce diagnosis time.

CASE STUDY 2: TRAFFIC SIGN RECOGNITION FOR AUTONOMOUS VEHICLES

Problem: Vehicles must recognize traffic signs in real time for safe navigation.

Solution: Computer vision detects and classifies traffic signs using feature extraction and CNN.

Impact: Prevents accidents and enables self-driving cars to follow traffic rules.



Real-World Use:

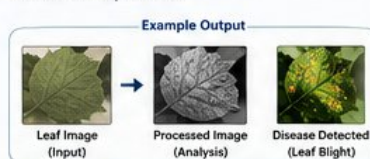
Used in Tesla, Waymo and many ADAS (Advanced Driver Assistance Systems).

CASE STUDY 3: CROP DISEASE DETECTION IN AGRICULTURE

Problem: Farmers cannot detect crop diseases early, leading to large losses.

Solution: Image processing analyzes leaf images and detects diseases.

Impact: Timely treatment, higher crop yield, reduced use of pesticides.



Real-World Use:

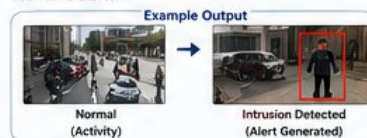
Used in mobile apps (e.g., Plantix) to help farmers identify crop diseases.

CASE STUDY 4: SECURITY SURVEILLANCE AND INTRUSION DETECTION

Problem: Manual monitoring of cameras is inefficient and cannot detect suspicious activities always.

Solution: Computer vision detects motion, unusual behavior and recognizes faces.

Impact: Improved safety, crime prevention and real-time alerts.



Real-World Use:

Used in airports, banks, smart cities and public places.

CASE STUDY 5: WASTE SEGREGATION USING COMPUTER VISION

Problem: Improper waste segregation causes pollution and health hazards.

Solution: Computer vision classifies waste (plastic, metal, paper, organic) using trained models.

Impact: Supports recycling, clean environment and smart waste management.



Real-World Use:

Used in smart bins and recycling plants for automatic waste sorting.

5. SOCIETAL IMPACT



Healthcare Improvement

Better diagnosis and patient care



Smart Cities

Traffic management, surveillance, smart infrastructure



Environmental Protection

Monitoring pollution, deforestation, wildlife protection



Industrial Growth

Quality control, automation, reduced human error



Education & Research

Helps in learning, data analysis and innovation



Better Everyday Life

Face unlock, photo enhancement, navigation, entertainment

6. FUTURE TRENDS

- ✓ Deep Learning & Neural Networks
- ✓ Edge AI and Real-Time Processing
- ✓ 3D Vision and LiDAR
- ✓ Multimodal AI (Image + Text + Audio)
- ✓ Explainable AI for Vision
- ✓ More Intelligent and Autonomous Systems



7. KEY TAKEAWAYS

- Image Processing improves image quality.
- Computer Vision understands and interprets visual data.
- These technologies solve real-world problems in multiple domains.
- They improve efficiency, safety and quality of life.
- The future will bring more intelligent and responsible AI-driven applications.



8. STUDENT ACTIVITY

Choose any one case study and:

- Find the real-world problem.
- Explain how image processing or computer vision solves it.
- Collect images and dataset (if available).
- Implement a basic solution using Python (OpenCV / Deep Learning).
- Present your results and impact.



Conclusion

Image Processing and Computer Vision are powerful technologies that convert visual data into meaningful information. They are transforming industries and solving many real-time challenges for a better and smarter world.

Tools & Libraries:

